KNN and Naive Bayes Example - Problem 7.2 & Problem 8.1 Personal Loan Acceptance

> (c) 2025 Galit Shmueli, Peter C. Bruce, Peter Gedeck
>
> Code included in
>
> *Machine Learning for Business Analytics: Concepts, Techniques, and Applications in Python* (Second Edition) Galit Shmueli, Peter C. Bruce, Peter Gedeck, and Nitin R. Patel. 2025.

Add comments to each **code** cell (no comments need to be added for Markdown).

Each code cell starts with two comments:

1. Your initials and last two digits of your ID number.
2. Provide a brief explanation of the code cell. If comments are already included in the code cell, you may revise them for clarity, summarize the existing comments, or leave them unchanged if they are already appropriate. For example,

JT88

This code cell imports necessary Python packages.

In [25]:
```
import pandas as pd
from sklearn import preprocessing
from sklearn.pipeline import Pipeline
from sklearn.model_selection import train_test_split, cross_validate
from sklearn.metrics import mean_squared_error
from sklearn.neighbors import KNeighborsClassifier, KNeighborsRegressor
import mlba
import math
```

Problem 7.2 Personal Loan Acceptance

Universal Bank is a relatively young bank growing rapidly in terms of overall customer acquisition. The majority of these customers are liability customers (depositors) with varying sizes of relationship with the bank. The customer base of asset customers (borrowers) is quite small, and the bank is interested in expanding this base rapidly to bring in more loan business. In particular, it wants to explore ways of converting its liability customers to personal loan customers (while retaining them as depositors).

A campaign that the bank ran last year for liability customers showed a healthy conversion rate of over 9% success. This has encouraged the retail marketing

department to devise smarter campaigns with better target marketing. The goal is to use k-NN to predict whether a new customer will accept a loan offer. This will serve as the basis for the design of a new campaign.

The dataset **UniversalBank.csv** contains data on 5000 customers. The data include customer demographic information (age, income, etc.), the customer's relationship with the bank (mortgage, securities account, etc.), and the customer response to the last personal loan campaign (Personal Loan). Among these 5000 customers, only 480 (= 9.6%) accepted the personal loan that was offered to them in the earlier campaign.

Partition the data into training (60%) and holdout (40%) sets.

7.2.a Consider the following customer:

Age = 40, Experience = 10, Income = 84, Family = 2, CCAvg = 2, Education_1= 0, Education_2 = 1, Education_3 = 0, Mortgage = 0, Securities Account = 0, CD Account = 0, Online = 1, and Credit Card = 1. Perform a k-NN classification with all predictors except ID and ZIP code using k = 1. Remember to transform categorical predictors with more than two categories into dummy variables first. Specify the success class as 1 (loan acceptance), and use the default cutoff value of 0.5. How would this customer be classified?

Answer: #VB67 Using k-NN with k =1, the customer is classified purely based on the class of the single nearest neighbor in the training data. Without the training dataset, the exact class (0 or 1) cannot be determined.

Data preparation

Load the data and remove unnecessary columns (ID, ZIP Code). Split the data into training (60%) and holdout (40%) sets (use `random_state=1`).

In [2]:
```
#VB67
#loading the dataset
bank_df = mlba.load_data('UniversalBank.csv')
# Drop unnecessary columns: 'ID' and 'ZIP Code' as they are not predictors
bank_df = bank_df.drop(columns=['ID', 'ZIP Code'])
# Display the first few rows of the dataframe
bank_df.head()
```

Out[2]:

	Age	Experience	Income	Family	CCAvg	Education	Mortgage	Personal Loan	Securiti Accou
0	25	1	49	4	1.6	1	0	0	
1	45	19	34	3	1.5	1	0	0	
2	39	15	11	1	1.0	1	0	0	
3	35	9	100	1	2.7	2	0	0	
4	35	8	45	4	1.0	2	0	0	

In [3]:

```
#VB67
# Replacing spaces and '=' signs in column names with underscores for easier
bank_df.columns = [c.replace(' ', '_').replace('=', '_') for c in bank_df.co
# Displaying the list of updated column names
list(bank_df.columns)
```

Out[3]:

```
['Age',
 'Experience',
 'Income',
 'Family',
 'CCAvg',
 'Education',
 'Mortgage',
 'Personal_Loan',
 'Securities_Account',
 'CD_Account',
 'Online',
 'CreditCard']
```

In [4]:

```
#VB67
bank_df.head()
```

Out[4]:

	Age	Experience	Income	Family	CCAvg	Education	Mortgage	Personal_Loan	Se
0	25	1	49	4	1.6	1	0	0	
1	45	19	34	3	1.5	1	0	0	
2	39	15	11	1	1.0	1	0	0	
3	35	9	100	1	2.7	2	0	0	
4	35	8	45	4	1.0	2	0	0	

In [5]:

```
#VB67
# Converting 'Education' column to categorical type
bank_df['Education'] = bank_df['Education'].astype('category')
# Creating dummy variables for all categorical columns, using '_' as prefix
bank_df = pd.get_dummies(bank_df, prefix_sep='_')
# Displaying the first few rows of the updated dataframe
bank_df.head()
```

Out[5]:

	Age	Experience	Income	Family	CCAvg	Mortgage	Personal_Loan	Securities_Ac
0	25	1	49	4	1.6	0	0	
1	45	19	34	3	1.5	0	0	
2	39	15	11	1	1.0	0	0	
3	35	9	100	1	2.7	0	0	
4	35	8	45	4	1.0	0	0	

In [6]:

```
#VB67
# Splitting the dataset into training and holdout sets (60% train, 40% holdo
train_df, holdout_df = train_test_split(bank_df, test_size=0.4, random_state
# Displaying the shapes of the training and holdout sets
print('Training set:', train_df.shape, 'Holdout set:', holdout_df.shape)
```

```
Training set: (3000, 14) Holdout set: (2000, 14)
```

In [7]:

```
#VB67
# Creating a DataFrame for a new customer with specified feature values
newCustomer = pd.DataFrame([{
    'Age': 40, 'Experience': 10, 'Income': 84, 'Family': 2, 'CCAvg': 2,
    'Mortgage': 0, 'Securities_Account': 0, 'CD_Account': 0, 'Online': 1,
    'CreditCard': 1,  'Education_1': False, 'Education_2': True, 'Education_
}])
# Displaying the new customer's data
newCustomer
```

Out[7]:

	Age	Experience	Income	Family	CCAvg	Mortgage	Securities_Account	CD_Acco
0	40	10	84	2	2	0	0	

In [8]:

```
#VB67
# Defining the target variable
outcome = 'Personal_Loan'
# Defining predictor variables (all columns except the outcome)
predictors = list(bank_df.columns)
predictors.remove(outcome)

# Creating a pipeline: standardize numeric predictors and apply k-NN classif
model = Pipeline([
    ('scaler', preprocessing.StandardScaler()),
    ('knn', KNeighborsClassifier(n_neighbors=5)),
])

# Separating predictors and target for training set
train_X = train_df[predictors]
train_y = train_df[outcome]
# Separating predictors and target for holdout set
holdout_X = holdout_df[predictors]
holdout_y = holdout_df[outcome]
```

In [9]:
```
#VB67
# Fitting the k-NN model on the training data
model.fit(train_X, train_y)

# Predicting the class for the new customer
print(f'Predicted class: {model.predict(newCustomer)}')
# Predicting the class probabilities for the new customer
print(f'Predicted probabilities: {model.predict_proba(newCustomer)}')
```

```
Predicted class: [0]
Predicted probabilities: [[1. 0.]]
```

New customer is predicted to not accept a loan offer.

7.2.b What is a choice of k that balances between overfitting and ignoring the predictor information? Use 5-fold cross-validation to find the best k.

Answer A choice of k that balances overfitting and ignoring predictor information is k = 3. This was determined using 5-fold cross-validation, which selects the k with the lowest prediction error.

In [10]:
```
#VB67
# Initialize an empty list to store cross-validation results
results = []
# Looping through k values from 1 to 19 to find the best number of neighbors
for k in range(1, 20):
    # setting the number of neighbors in the knn model
    model.set_params(knn__n_neighbors=k)
    accuracy = cross_validate(model, train_X, train_y, cv=5, scoring='accura
    # Store the results for each k
    results.append({'k': k, 'accuracy': accuracy})

# Converting the results list into a DataFrame for better readability
results = pd.DataFrame(results)
# Displaying the first 10 rows of results, rounded to 3 decimal places
results.round(3).head(10)
```

Out[10]:

	k	accuracy
0	1	0.961
1	2	0.958
2	3	0.964
3	4	0.957
4	5	0.961
5	6	0.954
6	7	0.957
7	8	0.953
8	9	0.955
9	10	0.950

In [11]:
```
#VB67
results.plot.scatter(x='k', y='accuracy')
```

Out[11]:
```
<Axes: xlabel='k', ylabel='accuracy'>
```

Our best k is k=3 which maximizes the accuracy based on 5-fold cross-validation accuracy scores computed on the training set.

7.2.c. Show the confusion matrix for the holdout data that results from using the best k.

Answer The confusion matrix for the holdout data using the best k=3 shows the model's classification performance. It summarizes the true positives, true negatives, false positives, and false negatives for loan acceptance prediction.

In [12]:
```
#VB67
# Set the optimal number of neighbors (k=3) in the k-NN model based on cross
model.set_params(knn__n_neighbors=3)
# Train (fit) the model using the training dataset
model.fit(train_X, train_y)
# Generate and display a classification summary comparing true vs. predicted
mlba.classificationSummary(y_true=holdout_y, y_pred=model.predict(holdout_X)
```

```
Confusion Matrix (Accuracy 0.9535)

       Prediction
Actual    0    1
     0 1797   10
     1   83  110
```

7.2.d. Consider the following customer: Age = 40, Experience = 10, Income = 84, Family = 2, CCAvg = 2, Education_1 = 0, Education_2 = 1, Education_3 = 0, Mortgage = 0, Securities Account = 0, CD Account = 0, Online = 1 and Credit Card = 1. Classify the customer using the best k.

Answer Using the best k=3, the customer is classified based on the majority class of the 3 nearest neighbors. The predicted class and probabilities can be obtained using model.predict(0) and model.predict_proba([1. 0.]), respectively.

In [13]:
```
#VB67
# Predicting the loan acceptance class (0 = No, 1 = Yes) for the new custome
print(f'Predicted class: {model.predict(newCustomer)}')
# Displaying the predicted probabilities for each class (No/Yes)
print(f'Predicted probabilities: {model.predict_proba(newCustomer)}')
```

```
Predicted class: [0]
Predicted probabilities: [[1. 0.]]
```

New customer is predicted to not accept a loan offer.

Problem 8.1 Personal Loan Acceptance.

The dataset **UniversalBank.csv** contains data on 5000 customers of Universal Bank. The data include customer demographic information (age, income, etc.), the customer's relationship with the bank (mortgage, securities account, etc.), and the customer response to the last personal loan campaign (Personal Loan). Among these 5000 customers, only 480 (= 9.6%) accepted the personal loan that was offered to them in the earlier campaign. In this exercise, we focus on two predictors: Online (whether or not the customer is an active user of online banking services) and Credit Card (abbreviated CC below) (does the customer hold a credit card issued by the bank), and the outcome Personal Loan (abbreviated Loan below).

Partition the data into training (60%) and holdout (40%) sets.

In [14]:
```
from sklearn.naive_bayes import MultinomialNB
```

Data Preparation

Remove all unnecessary columns from the dataset and convert *Online* and *CreditCard* to categories. Split the data into training (60%), and holdout (40%) sets (use `random_state=1`).

In [15]:
```
#VB67
# Loading the UniversalBank dataset
bank_df = mlba.load_data('UniversalBank.csv')
# Keeping only the relevant columns for analysis
bank_df = bank_df[['Online', 'CreditCard', 'Personal Loan']]
# Converting 'Online' column to categorical type
bank_df.Online = bank_df.Online.astype('category')
# Converting 'CreditCard' column to categorical type
bank_df.CreditCard = bank_df.CreditCard.astype('category')
```

```
# Displaying the first few rows of the updated DataFrame
bank_df.head()
```

Out[15]:

	Online	CreditCard	Personal Loan
0	0	0	0
1	0	0	0
2	0	0	0
3	0	0	0
4	0	1	0

Split dataset into training and holdout sets.

In [16]:

```
#VB67
# Splitting the dataset into training and holdout sets (60% training, 40% ho
train_df, holdout_df = train_test_split(bank_df, test_size=0.4, random_state
# Displaying the shape (rows, columns) of both the training and holdout sets
print('Training Set:', train_df.shape, 'holdout Set:', holdout_df.shape)
```

```
Training Set: (3000, 3) holdout Set: (2000, 3)
```

8.1.a Create a pivot table for the training data with Online as a column variable, CC as a row variable, and Loan as a secondary row variable. The values inside the table should convey the count. Use the pandas dataframe methods *pivot_table()*.

In [17]:

```
#VB67
# Creating a pivot table to summarize the relationship between CreditCard, F
pd.pivot_table(train_df, index=['CreditCard', 'Personal Loan'],
               columns=['Online'], aggfunc=len, observed=True)
```

Out[17]:

	Online	0	1
CreditCard	**Personal Loan**		
0	**0**	792	1117
	1	73	126
1	**0**	327	477
	1	39	49

8.1.b. Consider the task of classifying a customer who owns a bank credit card and is actively using online banking services. Looking at the pivot table, what is the probability that this customer will accept the loan offer? (This is the probability of loan acceptance (Loan = 1) conditional on having a bank credit card (CC = 1) and being an active user of online banking services (Online = 1)).

Answer:

Use the pivot table created in 8.1.b. for the answer

There are 477 + 49 = 526 records where online = 1 and cc = 1. 49 of them accept the loan, so the conditional probability is 49/526 = 0.0932

In [18]:
```
#VB67
# Calculating the conditional probability: P(Loan = 1 / CreditCard = 1, Onli
p11 = 49 / (477 + 49)
# Displaying the probability rounded to three decimal places
print(f'Count based probability P(Loan = 1|CC = 1, Online = 1) = {p11:.3f}')
```

```
Count based probability P(Loan = 1|CC = 1, Online = 1) = 0.093
```

8.1.c. Create two separate pivot tables for the training data. One will have Loan (rows) as a function of Online (columns) and the other will have Loan (rows) as a function of CC.

Answer

Pivot table for Loan (rows) as a function of Online (columns). Here we can use the `pivot_table` method of the pandas data frame.

In [19]:
```
#VB67
# Define the predictor variables
predictors = ['CreditCard', 'Online']

# Displaying the proportion of customers who accepted (1) or did not accept
print(train_df['Personal Loan'].value_counts() / len(train_df))
print()

# Loop through each predictor to examine its relationship with Personal Loan
for predictor in predictors:
    # construct the frequency table
    df = train_df[['Personal Loan', predictor]]
    freqTable = df.pivot_table(index='Personal Loan', columns=predictor, agg

    # divide each row by the sum of the row to get conditional probabilities
    propTable = freqTable.apply(lambda x: x / sum(x), axis=1)
    print(propTable)
    print()
```

```
Personal Loan
0    0.904333
1    0.095667
Name: count, dtype: float64

CreditCard            0         1
Personal Loan
0              0.703649  0.296351
1              0.693380  0.306620

Online                0         1
Personal Loan
0              0.412459  0.587541
1              0.390244  0.609756
```

8.1.d. Compute the following quantities, P(A | B) means "the probability of A given B"]:

- i. P(CC = 1 | Loan = 1) (the proportion of credit card holders among the loan acceptors)
- ii. P(Online = 1|Loan = 1)
- iii. P(Loan = 1) (the proportion of loan acceptors)
- iv. P(CC = 1|Loan = 0)
- v. P(Online = 1|Loan = 0)
- vi. P(Loan = 0)

Use the pivot tables created in 8.1.c.

- i. P(CreditCard = 1|Loan = 1) = 0.306620
- ii. P(Online = 1|Loan = 1) = 0.609756
- iii. P(Loan = 1) = 0.095667
- iv. P(CC = 1|Loan = 0) = 0.296351
- v. P(Online = 1|Loan = 0) = 0.587541
- vi. P(Loan = 0) = 0.904333

8.1.e. Use the quantities computed above to compute the naive Bayes probability P(Loan = 1 j CC = 1, Online = 1).

Refer to the naive Bayes formula (8.3) in the book.

```
P(Loan=1|CC=1,Online=1) =
   P(Loan=1) * P(CC=1|Loan=1) * P(Online=1|Loan=1) /
   [P(Loan=1) * [P(CC=1|Loan=1) * P(Online=1|Loan=1)] +
    P(Loan=0) * [P(CC=1|Loan=0) * P(Online=1|Loan=0)]]
```

In [20]:
```
#VB67
# Probability for Loan = 1 (accepted loan)
p1 = 0.095667 * 0.306620 * 0.609756
# Probability for Loan = 0 (did not accept loan)
```

```
p2 = 0.904333 * 0.296351 * 0.587541

# Compute the posterior probability using Bayes' theorem
# P(Loan=1|CC=1, Online=1) = p1 / (p1 + p2)
print(f'Naive Bayes probability P(Loan = 1|CC = 1, Online = 1) = {p1 / (p1 +
```

```
Naive Bayes probability P(Loan = 1|CC = 1, Online = 1) = 0.102
```

8.1.f. Compare this value with the one obtained from the pivot table in (b). Which is a more accurate estimate?

The value obtained from the crossed pivot table is the more accurate estimate, since it does not make the simplifying assumption that the probabilities (of taking a loan if you are a credit card holder and if you are an online customer) are independent. It is feasible in this case because there are few variables and few categories to consider, and thus there are ample data for all possible combinations.

8.1.g. In Python, run naive Bayes on the data. Examine the model output on the training data, and find an entry that corresponds to P(Loan = 1 | CC = 1, Online = 1). Compare this with the number you obtained in (e).

In [21]:
```
#VB67
# Converting all categorical columns in the training set into dummy/indicato
train_df = pd.get_dummies(train_df, prefix_sep='_')
# Converting the target variable 'Personal Loan' to categorical type
train_df['Personal Loan'] = train_df['Personal Loan'].astype('category')
# Displaying the first few rows of the updated training DataFrame
train_df.head()
```

Out[21]:

	Personal Loan	Online_0	Online_1	CreditCard_0	CreditCard_1
4522	0	True	False	True	False
2851	0	False	True	True	False
2313	0	False	True	False	True
982	0	True	False	False	True
1164	1	False	True	True	False

In [22]:
```
#VB67
# Defining predictor columns for the Naive Bayes model (dummy variables for
predictors = ['Online_0', 'Online_1', 'CreditCard_0', 'CreditCard_1']
# Initialize a Multinomial Naive Bayes classifier with smoothing parameter a
nb = MultinomialNB(alpha=0.01)
# Fitting the Naive Bayes model using the training data
nb.fit(train_df[predictors], train_df['Personal Loan'])
```

Out[22]:

```
▾ MultinomialNB
MultinomialNB(alpha=0.01)
```

In [23]:
```
#VB67
# Predicting class probabilities for all training instances using the fittec
predProb = nb.predict_proba(train_df.drop(columns=['Personal Loan']))
# Combining the original training DataFrame with the predicted probabilities
predicted = pd.concat([train_df, pd.DataFrame(predProb, index=train_df.index

# Filtering rows where Online = 1 and CreditCard = 1
matches = (predicted.Online_1 == 1) & (predicted.CreditCard_1 == 1)
# Displaying the first few rows that match the filter
predicted[matches].head()
```

Out[23]:

	Personal Loan	Online_0	Online_1	CreditCard_0	CreditCard_1	0	1
2313	0	False	True	False	True	0.897993	0.102007
1918	1	False	True	False	True	0.897993	0.102007
4506	0	False	True	False	True	0.897993	0.102007
586	0	False	True	False	True	0.897993	0.102007
3591	0	False	True	False	True	0.897993	0.102007

This gives `P(Loan=1|Online=1,CC=1) = 0.1020`

In [ ]: